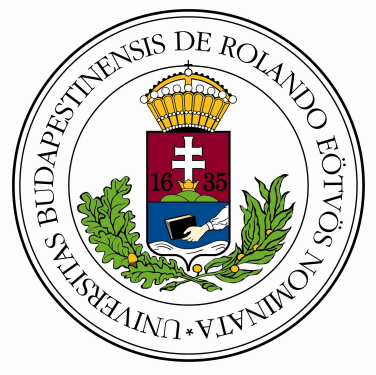
Konkurens programozás

Csatornaalapú kommunikáció



Kozsik Tamás

ELTE Eötvös Loránd Tudományegyetem

Termelő-fogyasztó probléma leprogramozása

- `java.util.concurrent.BlockingQueue`-val
- `java.io` streamekkel
  - `PipedInputStream` és `PipedOutputStream` (bájtszervezésű)
  - `PipedReader` és `PipedWriter` (karakterszervezésű)



Szálak közötti kommunikáció streamekkel

- `java.io.PipedInputStream` és `java.io.PipedOutputStream`
- `java.io.PipedReader` és `java.io.PipedWriter`

- Egy bemenetit és egy kimenetit összekapcsolunk
- A cső egyik végére az egyik szál ír, a cső másik végéről a másik szál olvassa
- Az olvasás blokkoló művelet!
  - `available()`, `ready()`



Előkészítés

```
import java.io.*;

...

PipedInputStream in = new PipedInputStream();
PipedOutputStream out = new PipedOutputStream(in);
```



Előkészítés

```
import java.io.*;

...

PipedInputStream in = new PipedInputStream();
PipedOutputStream out = new PipedOutputStream(in);

(new Producer(out)).start();
(new Consumer(in)).start();
```



Üzenetküldés

```
import java.io.*;
public class Producer extends Thread {

    private DataOutputStream out;

    Producer( OutputStream out ){
        this.out = new DataOutputStream(out);
    }

    public void run(){
        while( ... ){
            ...
            out.writeInt( ... ); out.flush();
            ...
        }
    }
}
```



Üzenetfogadás

```
import java.io.*;
public class Consumer extends Thread {

    private DataInputStream in;

    Consumer( InputStream in ){
        this.in = new DataInputStream(in);
    }

    public void run(){
        while( ... ){
            ...
            int message = in.readInt();
            ...
        }
    }
}
```



Oda-vissza kommunikáció

```
PipedInputStream   inA2B = new PipedInputStream(),
                   inB2A = new PipedInputStream();
PipedOutputStream outA2B = new PipedOutputStream(inA2B),
                  outB2A = new PipedOutputStream(inB2A);
(new MyThread(inB2A,outA2B)).start();   // thread A
(new MyThread(inA2B,outB2A)).start();   // thread B
```

```
public class MyThread extends Thread {
    private DataInputStream in;
    private DataOutputStream out;

    MyThread( InputStream in, OutputStream out ){
        this.in = new DataInputStream(in);
        this.out = new DataOutputStream(out);
    }
```

Kommunikációs csatorna

- Elosztott memóriás kommunikációs modell
- Üzenetküldés a folyamatok között

`java.util.concurrent.BlockingQueue`

`java.io.Piped*`

- hasonlít a socket-alapú kommunikációra
- egységes megoldás adható intra-JVM és inter-JVM kommunikációra



TCP-socket

```
import java.io.*;
import java.net.*;

...

void communicate( Socket socket ) throws IOException {
    DataInputStream in = new DataInputStream( socket.getInputStream() );
    DataOutputStream out = new DataOutputStream( socket.getOutputStream() );
    ...
    out.writeInt(42);
    out.flush();
    ...
    int n = in.readInt();
    ...
}
```



TCP-kliens

```
import java.io.*;
import java.net.*;

...

void communicate( Socket socket ) throws IOException { ... }
...
void startClient() throws IOException {
    try(
        Socket s = new Socket("localhost",1924)
    ){
        communicate(s);
    }
}
```



TCP-szerver

```
import java.io.*;
import java.net.*;

...

void communicate( Socket socket ) throws IOException { ... }
...
void startServer() throws IOException {
    try(
        ServerSocket ss = new ServerSocket(1924);
        Socket s = ss.accept();
    ){
        communicate(s);
    }
}
```



TCP-kliensek szekvenciális kiszolgálása

```
import java.io.*;
import java.net.*;

...

void communicate( Socket socket ) throws IOException { ... }
...
void startServer() throws IOException {
    try(
        ServerSocket ss = new ServerSocket(1924)
    ){
        while(true){
            try(
                Socket s = ss.accept();
            ){
                communicate(s);
            }
        }
}
```



Kliensenként új szál (JCIP)

```
public class ThreadPerTaskWebServer {
    public static void main(String[] args) throws IOException {
        ServerSocket socket = new ServerSocket(80);
        while (true) {
            final Socket connection = socket.accept();
            Runnable task = new Runnable() {
                public void run() {
                    handleRequest(connection);
                }
            };
            new Thread(task).start();
        }
    }

    private static void handleRequest(Socket connection) {
        // request-handling logic here
    }
}
```



Kliensenként egy feladat

```
public class TaskExecutionWebServer {
    private static final int NTHREADS = 100;
    private static final Executor exec
                        = Executors.newFixedThreadPool(NTHREADS);

    public static void main(String[] args) throws IOException {
        ServerSocket socket = new ServerSocket(80);
        while (true) {
            final Socket connection = socket.accept();
            exec.execute( () -> handleRequest(connection) );
        }
    }

    private static void handleRequest(Socket connection) {
        // request-handling logic here
    }
}
```



Szerver életcikluskezeléssel (JCIP)

```
public class LifecycleWebServer {
    private final ExecutorService exec = Executors.newCachedThreadPool();

    public void start() throws IOException {
        ServerSocket socket = new ServerSocket(80);
        while (!exec.isShutdown()) {
            try {
                final Socket connection = socket.accept();
                exec.execute( () -> handleRequest(connection) );
            } catch (RejectedExecutionException e) {
                if (!exec.isShutdown())
                    log("task submission rejected", e);
            }
        }
    }

    public void stop() {
        exec.shutdown();
    }
...
```

